Algoritmi un programmēšanas metodes(English)(1),25/26-P Mid-Term Assignment

Started on	Friday, 27 March 2026, 12:23 PM
State	Finished
Completed on	Saturday, 28 March 2026, 9:02 PM
Time taken	1 day 8 hours
Grade	**10.00** out of 10.00 (**100**%)

Information

Your task is to WRITE a program to find the biggest increase (difference) in a sequence of integers.

Unzip the following file pd1_data.zip: https://estudijas.rtu.lv/mod/resource/view.php?id=6366832 to obtain 7 files which need to be processed.

Integers are provided in a text format separated by newline.

Given a text file containing the following:

180

50

60

90

80

170

30

40

The answer would be 120 because the biggest increase is going from 50 to 170

Hint: start with a brute solution that we looked in the class

Hint2: there is a fairly simple linear solution to this

Hints: As a LAST resort, if you can't solve the original problem you can transform the problem to https://en.wikipedia.org/wiki/Maximum_subarray_problem ;

References: Cormen (CLRS) 4.1

JeffE: 3.3 - in class materials: https://jeffe.cs.illinois.edu/teaching/algorithms/book/03-dynprog.pdf

Question 1

Complete

Mark 0.50 out of 0.50

I certify the following:

Select one or more:

- [x] a. Source code for the program was written by myself.
- [x] b. Answers entered were generated by my own program
- [x] c. I've commented algorithmic portion of the code. If I used other's ideas/code, I've provided link/s in comments.

Question 2

Complete

Mark 1.00 out of 1.00

Algorithmic Complexity of my code is:

Select one:

- ○ a. Other
- ○ b. Θ(n^2)
- ○ c. Θ(n log n)
- ◉ d. Θ(n)

Question 3 Complete Mark 2.00 out of 2.00

Upload single file of your source code for this assignment.

Name should be

LastName_FirstName_PD1.ext

.ext will be the source code extension for the particular programming language that you use.

For example Saulespurens_Valdis_PD1.c

Allowed languages are:

C, C++, C#, Java, Scala, Kotlin, Javascript, PHP, Perl, Go, Rust, Python

IMPORTANT!!: Your code should be well commented especially the algorithmic portion

First comment should be your LastName FirstName PD1

It is **required** that you comment with links to sources to any found code fragments, algorithmic ideas.

```
# Ngoc Bao Tram Tran PD1
# 231ADB294

# Algorithmic Complexity: O(n)

import os # To work with file paths

def read_integers_from_file(file_path):
    """
    The pd1_data folder contains 7 text files.
    This function reads integers from a text file.
    Each line in the file contains one integer.
    The function returns a list of integers.
    """

    numbers = []

    # Read the file
    with open(file_path, 'r') as file:
        for line in file:
            line = line.strip() # Remove whitespace and newline characters
            if line: # Check if the line is not empty
                numbers.append(int(line)) # If the line is not empty, convert to integer and add to list
```

```
    return numbers # Return the predefined list

def highest_difference(numbers):
    """
    This function:
    Finds the biggest increase in the sequence.
    Increase is defined as numbers[j] - numbers[i], where j > i.
    Time complexity: O(n)
    """

    # If the list has fewer than 2 elements, so there is no difference can be computed
    if len(numbers) < 2:
        return 0

    # Initialize the smallest element - the first element of the list `numbers`
    minimum_element = numbers[0]

    # Initialize the maximum difference using first two elements of the list `numbers`
    maximum_difference = numbers[1] - numbers[0]

    # Loop through the list starting from index 1
    for i in range(1, len(numbers)):
        # Calculate difference between current element and minimum
        current_difference = numbers[i] - minimum_element

        # If this difference is larger, update maximum_difference
        if current_difference > maximum_difference:
            maximum_difference = current_difference

        # Update the minimum element if current value is smaller
        if numbers[i] < minimum_element:
            minimum_element = numbers[i]

    return maximum_difference


def main():
    file_path = os.path.join("pd1_data", "ints_10M.txt")
    file_name = os.path.basename(file_path)

    # Read all integers from the file
    numbers = read_integers_from_file(file_path)
```

```
    # Compute the highest difference from all integers in the file
    result = highest_difference(numbers)

    print("\t\t\tNgoc Bao Tram Tran PD1 - 231ADB294")
    print(f"The biggest increase (difference) in a sequence of integers in {file_name} is: {result}")

# References:
# https://www.geeksforgeeks.org/maximum-difference-between-two-elements/
# https://www.geeksforgeeks.org/python/os-module-python-examples/
# https://www.geeksforgeeks.org/dsa/analysis-algorithms-big-o-analysis/
```

main.py

Question 4 Complete Mark 0.50 out of 0.50

Enter highest increase for ints_10.txt

```
Increase is defined by the biggest difference n2-n1 where n2 comes after n1 in a particular file.
```

Answer: 46

Question 5 Complete Mark 1.00 out of 1.00

Enter highest increase for ints_100.txt

```
Increase is defined by the biggest difference n2-n1 where n2 comes after n1 in a particular file.
```

Answer: 962

Question 6

Complete

Mark 1.00 out of 1.00

Enter highest increase for ints_1K.txt

```
Increase is defined by the biggest difference n2-n1 where n2 comes after n1 in a particular file.
```

Answer: 9988

Question 7

Complete

Mark 1.00 out of 1.00

Enter highest increase for ints_10K.txt

```
Increase is defined by the biggest difference n2-n1 where n2 comes after n1 in a particular file.
```

Answer: 99971

Question 8

Complete

Mark 1.00 out of 1.00

Enter highest increase for ints_100K.txt

```
Increase is defined by the biggest difference n2-n1 where n2 comes after n1 in a particular file.
```

Answer: 999982

Question 9

Complete Mark 1.00 out of 1.00

Enter highest increase for ints_1M.txt

```
Increase is defined by the biggest difference n2-n1 where n2 comes after n1 in a particular file.
```

Answer: 9999964

Question 10

Complete Mark 1.00 out of 1.00

Enter highest increase for ints_10M.txt

```
Increase is defined by the biggest difference n2-n1 where n2 comes after n1 in a particular file.
```

Note: if you have Brute force solution, this could take a few days to run...

Answer: 99999960

◄ Final Group Project Exam - Spring 2026

Jump to...

Files to be processed for the mid-term 45MB zip ►